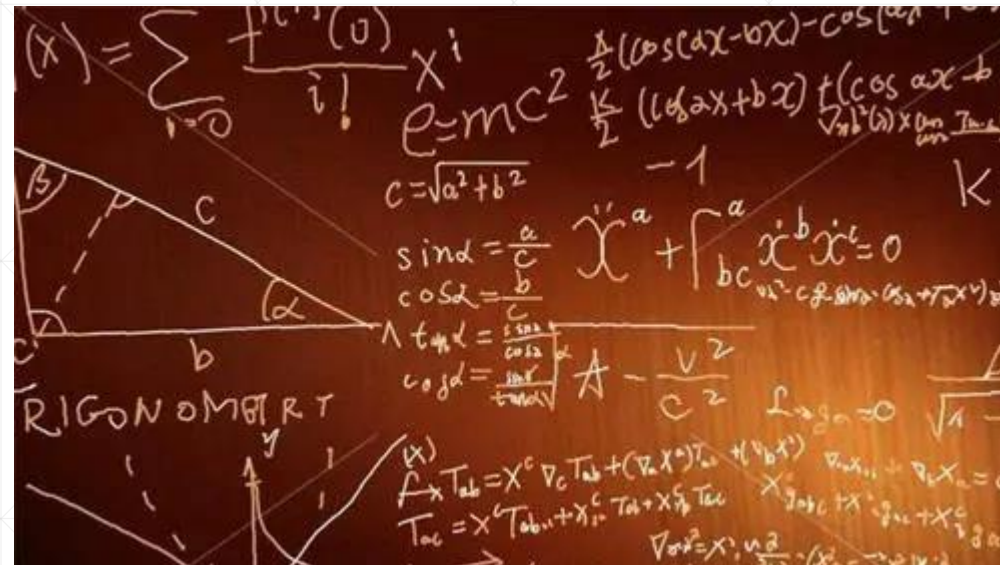




高阶函数

北京理工大学计算机学院
金旭亮

99

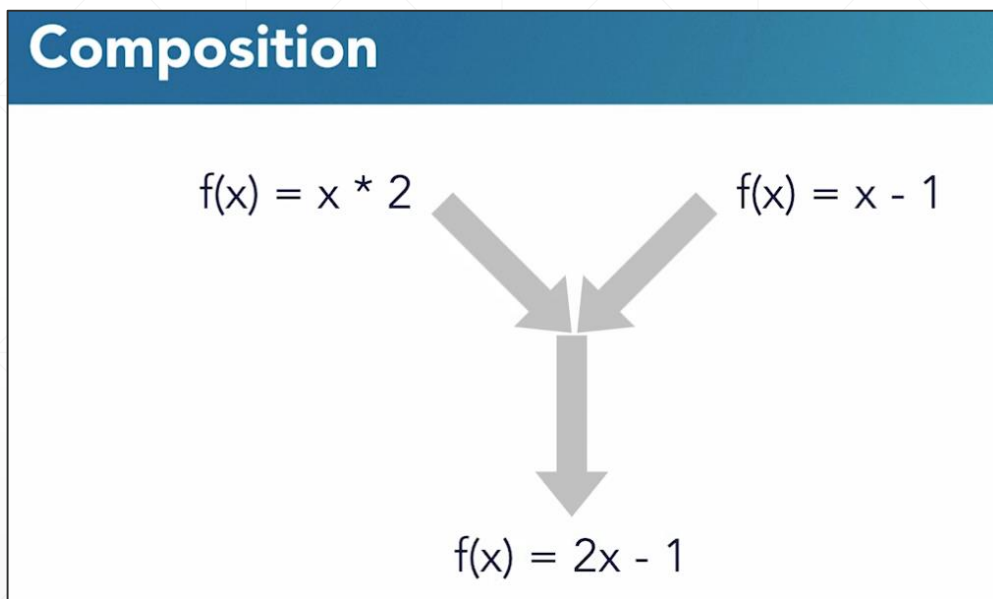
所谓高阶函数，就是接收函数作为参数，或者是返回一个函数的函数。

函数的复合



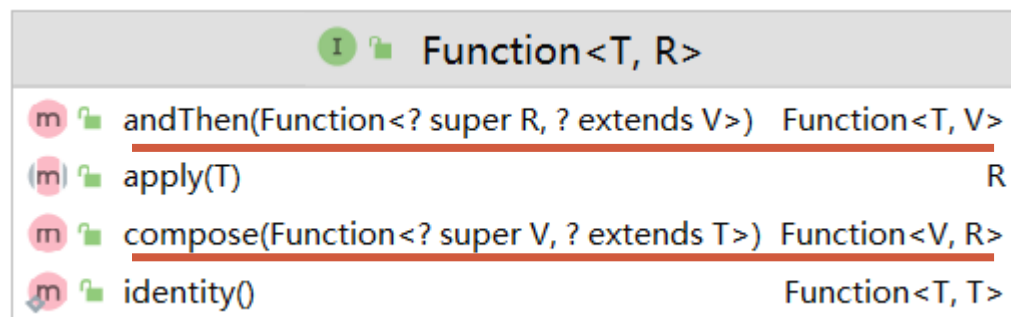
复合函数

数学中定义有一种“复合函数”：



Java实现函数复合

使用JDK所提供的Function<T,R>接口，可以很容易地实现函数的复合：

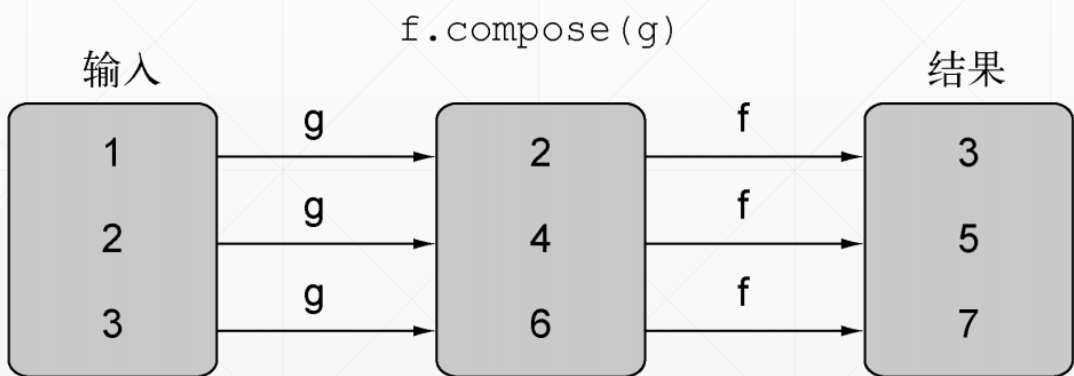
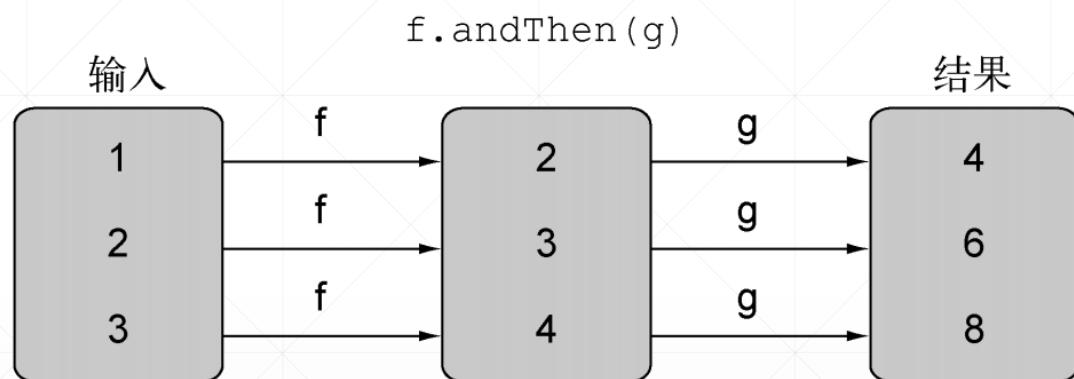


实现函数复合的关键在于`andThen`和`compose`方法，它们接收一个`Function`，又返回一个`Function`，这两个函数，都属于“高阶函数”。

理解函数的复合顺序



```
Function<Integer, Integer> f = x -> x + 1;  
Function<Integer, Integer> g = x -> x * 2;
```



注意`andThen`和`compose`两个方法合成函数的顺序是相反的。

复合函数示例

```
public class CompositionDemo {  
    public static void main(String[] args) {  
        Function<Integer, Integer> timesTwo = x -> x * 2;  
        Function<Integer, Integer> minusOne = x -> x - 1;  
        //函数的级联  
        Function<Integer, Integer> timesTwoMinusOne = timesTwo.andThen(minusOne);  
        System.out.println(timesTwoMinusOne.apply(10)); //19  
        //函数的合成 (先减后乘)  
        Function<Integer, Integer> composition = timesTwo.compose(minusOne);  
        System.out.println(composition.apply(10)); //18  
        //函数的合成 (先乘后减)  
        Function<Integer, Integer> composition2 = minusOne.compose(timesTwo);  
        System.out.println(composition2.apply(10)); //19  
    }  
}
```



andThen: 先买票, 后上车; compose: 先上车, 再买票。

多学一招



```
public class MyMathFunc {  
    public static Integer timesTwo(Integer x) {  
        return x * 2;  
    }  
  
    public static Integer timesThree(Integer x) {  
        return x * 3;  
    }  
  
    public static Integer timesFour(Integer x) {  
        return x * 4;  
    }  
  
    //通过返回一个函数，这个函数可以取代上述的三个函数  
    public static Function<Integer, Integer> createMultiplier(Integer y) {  
        return (Integer x) -> x * y;  
    }  
  
    public static void main(String[] args) {...}  
}
```

巧用高阶函数，可以减少不必要的定义。

```
public static void main(String[] args) {  
    Function<Integer, Integer> timesTwo = MyMathFunc.createMultiplier(y: 2);  
    Function<Integer, Integer> timesThree = MyMathFunc.createMultiplier(y: 3);  
    Function<Integer, Integer> timesFour = MyMathFunc.createMultiplier(y: 4);  
  
    System.out.println(timesTwo.apply(t: 6));  
    System.out.println(timesThree.apply(t: 6));  
    System.out.println(timesFour.apply(t: 6));  
}
```


函数的柯里化





99

所谓函数的“柯里化（Currying）”，通俗地说，就是把那些有多个参数的函数拆为多个参数个数较少的函数，通过级联调用这些函数，得到原先需要一次传入多个参数的效果。

实现“柯里化”的关键，在于精心设计函数的返回值，这个返回值必须又是一个函数，这样才能实现级联调用。

柯里化示例

定义一个接收三个参数的函数式接口：

```
@FunctionalInterface
public interface TriFunction<T, U, V, R> {
    R apply(T t, U u, V v);
}
```

用上述接口定义一个静态字段，接收相应的Lambda表达式：

```
static TriFunction<Integer, Integer, Integer, Integer> add =
    (x, y, z) -> x + y + z;
```

HighOrderDemo.java

参数拆分-1

```
//参数拆成1+2的形式
private static void demo1() {
    //定义一个返回BiFunction的高阶函数
    Function<Integer, BiFunction<Integer, Integer, Integer>> addPartial =
        (x) -> (y, z) -> add.apply(x, y, z);
    //先传入一个参数
    BiFunction<Integer, Integer, Integer> add1 = addPartial.apply(t: 1);
    //再传入剩余两个参数
    var result : Integer = add1.apply(t: 2, u: 3);
    //1+2+3=6
    System.out.println(result); //6
}
```

参数拆分-2

```
//参数拆成2+1的形式
private static void demo2() {
    //定义一个返回Function的高阶函数
    BiFunction<Integer, Integer, Function<Integer, Integer>> addPartial =
        (x, y) -> (z) -> add.apply(x, y, z);
    //先传入两个参数
    Function<Integer, Integer> add1And2 = addPartial.apply(t: 1, u: 2);
    //再传入剩余一个参数
    var result : Integer = add1And2.apply(t: 3);
    //1+2+3=6
    System.out.println(result); //6
}
```

参数拆分-2

```
//参数拆成1+1+1的形式
private static void demo3() {
    //嵌套三层的函数定义
    Function<Integer, Function<Integer, Function<Integer, Integer>>> myAdd =
    -----
        x -> (y) -> z -> x + y + z;
    //级联相加
    var result : Integer = myAdd.apply(t: 1).apply(t: 2).apply(t: 3);
    System.out.println(result);
}
```


小结



本讲主要介绍了两个重要的函数式编程技巧——函数的合成与柯里化。



在实际开发中，函数的合成还时不时地用到，而柯里化，用到的场景实在不多，所以，对于此技巧，了解即可。



至此，有关Lambda与函数式编程相关的理论知识与编程技巧都介绍完了，下一讲，将进入函数式编程的应用领域，介绍JDK中Stream API的用法。